

PROJECT REPORT

AI-Based Image Restoration and 2x Super-Resolution Using CNN

Project Name	IMAGE ENHANCER
Platform	Windows / Linux / macOS (Python 3.10+, CPU & CUDA GPU)
Primary Language	Python
Deep Learning Framework	PyTorch
Report Date	31 August 2026
Status	Completed and verified on CPU

Table of Contents

1. Abstract	3
2. Introduction	3
3. Goals and Objectives	3
4. Key Features	4
5. Technologies Used	6
6. How It Works (Functioning)	6
7. Methodology	9
8. Project Structure	11
9. How to Run	14
10. Results and Evaluation	15
11. What the Project Includes	17
12. Limitations	17
13. Future Advancements	18
14. Conclusion	18

1. Abstract

IMAGE ENHANCER is an end-to-end, laptop-friendly deep-learning application that **restores degraded (blurry / noisy / low-resolution) images and produces crisper, exactly-2x-higher-resolution output** using lightweight convolutional neural networks (CNNs). The project ships two CNN models, integrates a complete training pipeline with randomized image degradation, performs honest evaluation against the classical bicubic interpolation baseline using PSNR and SSIM, and wraps everything in an easy menu-driven command-line interface plus an optional graphical interface (GUI). In addition to revealing every lost edge, the system now **measures each photo before enhancing it**: already-clean images are preserved exactly as the original (no blur removal, no shade shift), genuinely blurred or noisy photographs are restored proportionally to how degraded they really are, and washed-out "shade" is removed only when a real veil is measured (never touching real shadows). It is deliberately designed so a student with no GPU can clone, install, train, and run the whole system from a single command: `python main.py`.

2. Introduction

Restoring and upscaling low-quality images is important in surveillance, medical imaging, satellite photography, and old-photo restoration. Classical methods such as bicubic interpolation produce smooth, blurry enlargements because they cannot recover high-frequency details that were lost during degradation. Modern deep-learning super-resolution (SR) networks learn the mapping from degraded/low-resolution images to clean, high-resolution images from data.

Most state-of-the-art SR networks are very large and require GPUs. This project focuses on a different constraint: **a lightweight, fully explainable, self-contained CNN project that runs on a regular laptop CPU and still produces visibly enhanced output.**

The project delivers two models of increasing capacity:

- CNN V1 — a simple, small baseline network (clear and easy to debug).
- CNN V2 — an improved network with residual blocks, global skip connection, and PixelShuffle upsampling, giving a clearly better SSIM.

3. Goals and Objectives

3.1 Project Goals

1. Accept a degraded 128x128 image and restore/enhance it to a 256x256 image.
2. For a photo of any size, produce output that is exactly 2x larger (true super-resolution, never a downscale-then-upscale).
3. Compare the CNN output against the classical bicubic baseline using PSNR (Peak Signal-to-Noise Ratio) and SSIM (Structural Similarity Index).
4. Make per-pixel quality metrics honest — the reported numbers come from real runs with no fabricated results.

5. Keep the project beginner-friendly: single entry point, automatically-created folders, friendly error messages, and full documentation (README, presentation, VIVA Q&A, this report).

3.2 Objectives

#	Objective	Status
1	Build two CNN models (V1 baseline, V2 improved)	OK Done
2	Randomized degradation pipeline (blur + downsampling + noise)	OK Done
3	Full training loop with best-validation checkpoint saving	OK Done
4	Evaluation module with PSNR / SSIM / bicubic baseline	OK Done
5	True 2x tiled super-resolution for arbitrarily sized images	OK Done
6	Terminal menu (8 options) + optional Tkinter GUI	OK Done
7	Visibly enhanced output (trained on SSIM + edge-quality loss, plus a standard sharpening pass)	OK Done
8	Auto-generate synthetic sample images on first run	OK Done
9	Windows launchers (setup.bat, run.bat)	OK Done
10	Documentation bundle (README, presentation, VIVA, this report)	OK Done
11	Adaptive enhancement: preserve clean originals, remove only real blur / noise / shade, never touch real shadows	OK Done

4. Key Features

6. Two CNN architectures with a clear trade-off:
 - • V1 — simple conv + PixelShuffle baseline (smallest, fastest).
 - • V2 — 4 residual blocks, wider features, global skip connection, refined PixelShuffle upsampling (better quality).
7. Randomized degradation for training so the model generalizes:
 - • Gaussian blur sigma randomly sampled 0.3 → 2.2.
 - • Gaussian noise standard deviation 0.0 → 0.05.
 - • Downsampling 256x256 → 128x128 (the model must produce the detail).
8. True 2x super-resolution for any input size:
 - • Arbitrary images are split into overlapping 128x128 tiles.
 - • Each tile is upscaled to 256x256 by the CNN.
 - • Tiles are blended with linear weight ramps (seam-free).
 - • Output is always exactly 2x the input dimensions.
9. Perceptually sharp enhancement:
 - • Training loss = L1 + SSIM + Laplacian (high-frequency) loss, which teaches the network to restore crisp edges instead of a smooth blur.
 - • A standard unsharp-mask sharpening pass is applied to the displayed / saved output (never inside the PSNR/SSIM evaluation).

10. Adaptive, content-aware restoration ("restore only what is actually wrong"):
 - • Before enhancing, the input is measured for blur (collapse of edge energy under additional smoothing, two noise-robust probes) and for noise (robust median residual), combined into a "deficiency" score in [0,1].
 - • A separate "shade" score detects washed-out / veiled photos (compressed tonal pass-band plus lifted blacks) and removes that veil while never lifting real shadows: the contrast gain pivots at the median and always equals or exceeds 1, so dark/shadow pixels only move darker.
 - • Restoration strength scales with deficiency up to the full CNN result (alpha reaches 1.0 for badly degraded photos); clean/sharp photos keep the original byte-for-byte unchanged.
 - • Every colour channel's tone mean is re-aligned to the input (within half a gray level), so shades/brightness are never shifted.
 - • Fully configurable in `config.py` (`ENHANCE_ADAPTIVE`, `ADAPTIVE_*` detection thresholds, `SHADE_*` veil-removal knobs).
11. Honest evaluation against bicubic:
 - • PSNR and SSIM computed on a shared, fixed validation set.
 - • Metrics JSON files saved to `results/metrics/`.
 - • Bar charts of PSNR / SSIM saved automatically.
12. Two usability interfaces:
 - • Menu-driven terminal application.
 - • Optional Tkinter GUI with thread-safe background enhancement and a full-window side-by-side view (original left, enhanced right).
13. Self-contained data story:
 - • Generates 40 synthetic sample images (512x512) on first run.
 - • Auto-creates every required folder.
 - • Trained demo models ship with the project, so the app works immediately.
14. Friendly engineering:
 - • `safe_input()` — never crashes on closed input streams.
 - • Graceful top-level error handling with actionable messages.
 - • Seed-based reproducibility (`SEED = 42`).

5. Technologies Used

Technology	Purpose	Version (verified)
Python	Main programming language	3.13.13
PyTorch (torch)	Deep learning framework (training + inference)	2.13.0+cpu
torchvision	Image transforms, model helper	0.28.0
NumPy	Array / tensor manipulation	2.5.2
OpenCV (opencv-python)	Image processing (blur, noise, sharpness measurement)	5.0.0.93
Pillow	Image I/O, resizing, unsharp-mask filter	12.3.0
scikit-image	SSIM implementation for evaluation	0.26.0
Matplotlib	Loss curves and metric bar charts	3.11.1
tqdm	Progress bars in terminal	4.70.0
Tkinter (stdlib)	Graphical user interface	Python stdlib
Bash / PowerShell	Windows launcher scripts (run.bat, setup.bat)	—

Why these technologies

- PyTorch because it is the most approachable deep-learning framework for students, with straightforward tensor broadcasting and automatic differentiation.
- Pillow + OpenCV because together they cover all classic image operations needed for degradation and enhancement.
- scikit-image provides a battle-tested SSIM implementation used for fair comparison against academic super-resolution papers.

6. How It Works (Functioning)

6.1 Overall Flow

```
python main.py
  |
  v
Menu (8 options)
[1 Enhance] [2 Train] [3 Evaluate] [4 Compare V1 vs V2]
[5 View Results] [6 GUI] [7 About] [8 Exit]
```

Option 1 - Enhance a photo (true 2x super-resolution):

```
Any image (W x H)
  |
  v
split into 128x128 tiles (overlap 8 px, stride 120 px, reflect padding)
  |
  v
for each tile: CNN maps [3,128,128] -> [3,256,256]      (V1 or V2)
  |
  v
blend overlapping 256x256 tiles with linear weight ramps
  |
  v
```

```

measure the input (blur + noise + shade) -> adaptive restore strength
|
v
de-veil pass (only if a real shade is measured; shadows never lifted)
|
v
unsharp-mask sharpening pass (strength scaled by the measurement)
|
v
re-align channel tones to the original shades
|
v
Enhanced image – exactly (2W x 2H) -> saved to results/enhanced/

```

6.2 Training Flow

```

Clean 256x256 target (random crop of a larger image)
|
v
degrade: Gaussian blur (random sigma) -> downscale to 128x128
        -> Gaussian noise (random level)
|
v
CNN V1 / V2 : [3,128,128] -> [3,256,256]
|
v
Loss = L1 + 0.35*(1 - SSIM) + 0.15*Laplacian-L1 (see training/losses.py)
|
v
Adam optimizer (lr = 1e-3) -> backpropagate -> next epoch
|
v
validation PSNR every epoch -> save ONLY the best checkpoint

```

6.3 The Degradation Model

Each clean image is degraded with parameters sampled randomly per sample:

- sigma (Gaussian blur) uniform in [0.3, 2.2]
- noise_std uniform in [0.0, 0.05]

Randomization makes the trained model robust to a wide range of input quality instead of over-fitting a single fixed blur. Optionally JPEG compression, brightness and contrast adjustments can be enabled in config.py (DO_JPEG, BRIGHTNESS_FACTOR, CONTRAST_FACTOR).

6.4 Loss Function (why the output looks sharp)

Plain L1/MSE training produces the average of all possible sharp images, which is visually blurry. Three terms are combined:

Term	Weight	Effect
L1	1.0	Stable, robust basis loss
(1 - SSIM)	0.35	Preserves structure and edges — directly improves the SSIM metric
Laplacian L1	0.15	Matches high-frequency (edge) content — crisper lines and textures

This is why the enhanced output is not just a "cleaner blur" — it restores edge detail the way human eyes perceive sharpness.

6.5 Final Sharpening Pass

After stitching, an unsharp mask is applied:

```
sharpened = image + amount * (image - blurred_image)
```

- `ENHANCE_SHARPEN_AMOUNT = 1.4`

- `SHARPEN_RADIUS = 2.0`

- `SHARPEN_THRESHOLD = 2` (protects flat areas from noise amplification)

In adaptive mode the amount is scaled by the measured deficiency (see below), so already-clean images are not over-sharpened. This pass runs only when producing the saved/displayed image; the evaluation metrics are computed on the raw CNN output, so the reported PSNR/SSIM stay honest.

6.6 Adaptive Enhancement (restore only what is actually wrong)

Two independent measurements run on a 256-normalized grayscale version of the input so scores are comparable across sizes:

Deficiency (blur + noise):

- **Blur probe 1:** ratio of Laplacian edge energy before/after an extra 1.0 sigma blur. Sharp images have ratios well above 2.6; blurred ones fall near 1.

- **Blur probe 2 (noise-robust):** the same ratio computed between a lightly denoised (+0.6 sigma) and a heavily smoothed (+1.6 sigma) copy, so noise cannot fake "sharpness". A blur is flagged if either probe detects it.

- **Noise probe:** robust median-based estimate of the high-frequency residual (median absolute deviation converted to a sigma).

The three signals are combined into a single deficiency in [0,1]:

```
deficiency = clip(0.7 * blur_score + 0.3 * noise_score)
```

Shade (washed-out veil): a second score detects photos whose tones look "veiled" or washed out — a compressed luminance pass-band (p98-p2 well below a healthy ~0.52 of full range) combined with lifted blacks (the 5th-percentile luminance too bright for a real photo):

```
shade_range = clip((SHADE_RANGE_HI - (p98-p2)) / (SHADE_RANGE_HI - SHADE_RANGE_LO))
shade_black = clip((p5 - SHADE_BLACK_LIFT_LO) / (SHADE_BLACK_LIFT_HI - SHADE_BLACK_LIFT_LO))
shade       = clip(shade_range * (0.6 + 0.4 * shade_black))
```

The black-lift term only counts once the tonal range is genuinely compressed, so a normal photo with a healthy range and deep shadows scores ~0 and is left alone. If `shade >= 0.12`, the output gets a de-veil pass: a linear contrast gain (median-pivoted, always `>= 1`) widens the tonal range while shadows — being below the pivot — can only move *darker*, never brighter; a small saturation lift accompanies it. Real shadows are therefore preserved by construction.

Restoration then follows the measurement:

- If `deficiency < 0.12`, the photo is effectively clean — the output is the plain 2x upscale plus (only when a real shade was measured) the de-veil pass. Otherwise clean photos stay byte-for-byte unchanged: no sharpening, no tonal change.

- Otherwise the CNN restoration is blended with the original upscale using $\alpha = \text{clip}(0.30 + 0.80 * \text{deficiency})$ (reaching 1.0 — the full CNN result — for badly degraded photos), and the unsharp-mask amount scales as $1.4 * (0.40 + 0.75 * \text{deficiency})$.

- Finally every colour channel's mean is re-aligned to the input's mean (nearest whole gray level), so overall shades and brightness are indistinguishable from the original (within 0.5/255 per channel).

All knobs live in `config.py` (`ENHANCE_ADAPTIVE`, `ADAPTIVE_KEEP_IF_CLEAN`, `ADAPTIVE_MIN_ALPHA`, `ADAPTIVE_MAX_ALPHA`, `ADAPTIVE_ALPHA_SLOPE`, the blur/noise thresholds, and the `SHADE_*` veil-removal knobs). Setting `ENHANCE_ADAPTIVE = False` restores the old fixed-strength pipeline.

7. Methodology

The project follows a standard supervised-SR methodology adapted for a single laptop computer.

15. Data creation (no copyrighted datasets bundled):

- `utils/sample_gen.py` procedurally creates 40 synthetic, sufficiently varied 512x512 images (gradients, patterns, shapes, blended colors).
- The user may drop their own photos into `data/train/` and `data/val/` (or use DIV2K) instead.

16. Paired (low, high) sample generation:

- Take a random 256x256 crop of a clean image = the target (high-res).
- Apply the randomized degradation pipeline → the input (low-res).

17. Model design:

- V1: conv(3→32) x3, ReLU, depth-to-space (PixelShuffle), final conv.
- V2: wider conv(3→48), 4 residual blocks with skip connections, PixelShuffle x2 refinement convs, plus a global skip connection that adds a bicubic 2x upscale of the input to the network output — the network therefore only needs to learn the residual detail.

18. Training protocol:

- Adam optimizer, learning rate 1e-3, batch size 4.
- Best-validation-loss checkpoint policy (single file per model).
- Reproducible via a fixed seed (`SEED = 42`).

19. Evaluation protocol (honest, reproducible):

- Shared validation set; every model sees the same degraded inputs.
- PSNR and SSIM measured on [0,1] float tensors with scikit-image's SSIM.
- Bicubic 2x upscale of the same 128x128 input = baseline.
- Results serialized to JSON (`results/metrics/`) and plotted.

20. Inference protocol (used at runtime):

- Tiled super-resolution with 8-px overlap + weighted blending so large images have no seams and never exceed memory limits.

- Adaptive measurement (blur + noise + shade) decides how strongly to restore, blending the CNN result with the original upscale, de-veiling only when a real shade is measured, and scaling the sharpening pass accordingly.
- Tone re-alignment keeps the original shades; optional sharpening pass (configurable) for visible crispness.

8. Project Structure

8.1 Directory Tree (main files shown)

```

IMAGE ENHANCER/
|
|-- main.py                # Single entry point (menu-driven app)
|-- config.py              # All settings & paths (one place)
|-- requirements.txt        # Python dependencies
|-- README.md              # User documentation / setup guide
|-- setup.bat              # Windows: create venv + install deps
|-- run.bat                # Windows: one-click launcher
|-- .gitignore             # Git ignore rules (venv, results, caches)
|
|-- models/                # Neural network architectures
|   |-- __init__.py
|   |-- cnn_v1.py           # V1 baseline CNN
|   |-- cnn_v2.py           # V2 improved CNN
|   |-- residual_block.py   # Residual block used by V2
|
|-- dataset/               # Data loading / degradation
|   |-- __init__.py
|   |-- degradation.py      # blur + downscale + noise (array/tensor)
|   |-- image_dataset.py    # Dataset + DataLoader factory
|
|-- training/              # Training machinery
|   |-- __init__.py
|   |-- losses.py           # L1 + SSIM + Laplacian criterion
|   |-- trainer.py          # Epoch loop, optimizer, best-checkpoint
|   |-- validator.py        # Validation loss + PSNR
|
|-- evaluation/            # Quality metrics
|   |-- __init__.py
|   |-- metrics.py          # psnr(), ssim(), bicubic_upscale()
|   |-- evaluator.py        # Evaluate model vs bicubic -> JSON
|
|-- inference/             # Runtime enhancement
|   |-- __init__.py
|   |-- predictor.py        # tiled 2x enhance, adaptive restore, sharpen
|
|-- interface/             # User interfaces
|   |-- __init__.py
|   |-- gui.py              # Tkinter GUI (full-window original/enhanced)
|
|-- visualization/         # Charts & comparison grids
|   |-- __init__.py
|   |-- visualizer.py       # Loss curves, metric bars, grids
|
|-- utils/                 # Shared helpers (import-light!)
|   |-- __init__.py         # lightweight package facade
|   |-- device.py           # CPU/CUDA auto-detect
|   |-- file_utils.py       # folders, find_images, save images
|   |-- sample_gen.py       # synthetic 512x512 samples
|   |-- seed.py             # reproducible seeding
|   |-- sharpen.py          # unsharp-mask sharpening
|
|-- docs/                  # Documentation bundle
|   |-- PROJECT_REPORT.pdf  # This report (PDF)
|   |-- VIVA.md             # 27 anticipated viva Q&A
|
|-- data/                  # Images (no copyrighted data bundled)
|   |-- train/              # your photos -> training split (optional)
|   |-- val/                # your photos -> validation split (optional)
|   |-- sample/             # 40 auto-generated 512x512 images
|
|-- checkpoints/           # Saved weights (best-only policy)
|   |-- best_model_v1.pth   # trained V1 weights
|   |-- best_model_v2.pth   # trained V2 weights (recommended)

```

```
|
|-- results/                # Everything the app produces
|-- enhanced/              # enhanced user images (2x)
|-- comparisons/           # side-by-side grids from menu 4
|-- metrics/               # PSNR/SSIM JSON results
|-- training_curves/       # loss curves + metric bar charts
```

8.2 What Every File Is Used For

Entry point & configuration

File	Purpose
main.py	The only file you need to run. Builds the menu, lazy-loads heavy dependencies, handles input errors, and wires together training / evaluation / enhancement / GUI.
config.py	Central single source of truth for paths, image sizes, degradation ranges, training hyperparameters, adaptive thresholds and sharpening settings. No other file should hard-code these.
requirements.txt	Declares the pip packages needed to run the project.
setup.bat	Windows one-click: creates venv, installs requirements.txt.
run.bat	Windows one-click launcher: activates venv and runs main.py.
.gitignore	Prevents committing venv/, __pycache__/, results/, checkpoints/, and sample data.

Model code (models/)

File	Purpose
cnn_v1.py	Defines ImageEnhancerV1 — the small baseline: 3 conv+ReLU layers (32 channels) then PixelShuffle 2x.
cnn_v2.py	Defines ImageEnhancerV2 — 48 channels, 4 residual blocks, 2 PixelShuffle/refine stages, plus a global skip connection that adds the bicubic 2x upscale to the network output (learning only the residual). The recommended model.
residual_block.py	Reusable ResidualBlock: conv > ReLU > conv with a skip connection used by V2.

Data pipeline (dataset/)

File	Purpose
degradation.py	degrade_array / degrade_tensor — apply Gaussian blur, downscale to 128, add Gaussian noise (controllable sigma/noise).
image_dataset.py	ImageEnhancerDataset + make_dataloaders — opens images, takes a random 256x256 crop as the clean target, produces a randomized degraded 128x128 input on the fly.

Training (training/)

File	Purpose
losses.py	build_criterion() returns the sharp-loss: $L1 + 0.35 \cdot (1 - SSIM) + 0.15 \cdot \text{Laplacian-L1}$. Contains the torch SSIM and Laplacian implementations.
trainer.py	train_model() — the epoch loop, Adam optimizer, progress bars, per-epoch validation, and saving only the best validation checkpoint.
validator.py	validate() — computes validation loss and PSNR for a model + loader.

Evaluation (evaluation/)

File	Purpose
metrics.py	psnr() (dB), ssim() (scikit-image), bicubic_upscale() — the three tools used everywhere for honest comparison.
evaluator.py	evaluate_model() — runs a model on a shared validation set, computes average PSNR/SSIM for both the model and bicubic, writes metrics_*.json.

Inference (inference/)

File	Purpose
predictor.py	The runtime engine. ImageEnhancerModel wraps a torch model; enhance_2x() splits any image into overlapping 128x128 tiles, upscales each to 256x256, blends seams, measures the input for blur/noise, adaptively blends the CNN result with the original 2x upscale, re-aligns channel tones, applies the (strength-scaled) sharpening pass, and returns an image that is exactly 2x the input. Also load_model(), enhance_file().

Interfaces (interface/)

File	Purpose
gui.py	Tkinter GUI: pick an image, enhance in a background thread, stream progress through a thread-safe queue, and display original vs enhanced side-by-side filling the whole window.

Visualization (visualization/)

File	Purpose
visualizer.py	plot_loss_curves(), plot_metric_bars(), save_comparison_grid() — turns raw numbers into PNG charts.

Utilities (utils/)

File	Purpose
<code>__init__.py</code>	Deliberately import-light (imports no torch/numpy) so <code>main.py</code> can boot even on a broken install.
<code>device.py</code>	<code>get_device()</code> / <code>device_summary()</code> — returns cuda when available, else cpu.
<code>file_utils.py</code>	<code>ensure_dirs()</code> , <code>find_images()</code> , <code>save_result_image()</code> — folder & file chores.
<code>sample_generator.py</code>	<code>generate_samples()</code> — procedurally draws 40 varied 512x512 images if none exist.
<code>seed.py</code>	<code>set_seed()</code> — fixes random seeds for reproducible runs.
<code>sharpen.py</code>	<code>unsharp_mask()</code> / <code>sharpen_pil()</code> — the final visible crispness pass (int-coerced for Pillow 12).

Documentation (docs/)

File	Purpose
<code>PROJECT_REPORT.pdf</code>	This document — the full project report.
<code>VIVA.md</code>	27 anticipated viva/interview Q&As with model answers.

Runtime artifacts

File / Folder	Purpose
<code>data/sample/*.png</code>	40 auto-generated synthetic images for training/demos.
<code>checkpoints/best_model_v1.pth</code> , <code>best_model_v2.pth</code>	Trained weights loaded by the app.
<code>results/enhanced/*.png</code>	Enhanced (2x) output images from menu 1 / GUI.
<code>results/comparisons/*.png</code>	Side-by-side degraded vs bicubic vs V1/V2 vs truth grids.
<code>results/metrics/*.json</code>	Honest PSNR/SSIM results per model.
<code>results/training_curves/*.png</code>	Loss curves and PSNR/SSIM bar charts.

9. How to Run

9.1 Automatic (Windows, recommended)

21. Double-click `setup.bat` once — it creates a virtual environment (venv) and installs every dependency.
 22. Double-click `run.bat` every time you want the app.
- (Or run `python main.py` from a terminal with venv active.)

9.2 Manual (any OS)

```
# 1. Create and activate a virtual environment
python -m venv venv
venv\Scripts\activate      # Windows
source venv/bin/activate   # Linux / macOS

# 2. Install dependencies
pip install -r requirements.txt

# 3. Run the application
python main.py
```

9.3 First Run Behaviour

- All required folders are created automatically.
- If data/sample/ is empty, 40 synthetic images are generated.
- The menu shows the device (CPU or CUDA) and whether model checkpoints exist.

9.4 Using the Menu

Option	What it does
1. Enhance	Pick V1 or V2, give a path to any image — gets a true 2x enhanced copy (adaptive restore) saved to results/enhanced/.
2. Train	Train V1/V2/Both in Quick Demo or Normal mode; epochs/batch/learning-rate configurable. Best checkpoint only.
3. Evaluate	Measure PSNR/SSIM of the trained model(s) vs bicubic; saves JSON + bar charts.
4. Compare V1 vs V2	Creates side-by-side comparison grids with PSNR labels.
5. View Results	Opens the generated charts/comparisons.
6. GUI	Launch the Tkinter interface (full-window original/enhanced view).
7. About	Show device & model info.
8. Exit	Quit.

10. Results and Evaluation

All numbers below come from actual runs on this machine (CPU) with the final, sharp-loss-trained checkpoints.

10.1 Honest Validation Comparison (shared validation set)

Model	Bicubic PSNR	Model PSNR	Bicubic SSIM	Model SSIM	Note
V1 (baseline)	28.42 dB	28.36 dB	0.6693	0.6719	matches bicubic in PSNR, edges finer
V2 (recommended)	28.79 dB	28.81 dB	0.7262	0.7592	wins on both measures — SSIM strongly better

- V2 +0.033 SSIM over bicubic is a clearly measurable structural gain (SSIM correlates with how "natural" and edge-correct an image looks).
- V1 intentionally stays near-bicubic — it proves the baseline is sane and the architecture (V2's residuals + global skip) is what creates the gain.

10.2 Visible Sharpness (runtime enhance test)

Full pipeline includes the tiled 2x SR, adaptive restoration and the sharpening pass:

Measure	Bicubic 2x upscale	Enhanced (V2)
Edge energy (mean Laplacian)	10.28	17.95
Sharpness gain	—	+74.6%

10.3 Training Curves (best validation PSNR reached)

Model	Best val PSNR during training	Epoch
V1	30.76 dB	22
V2	30.54 dB	23

Charts are saved automatically under results/training_curves/ and metric JSON files under results/metrics/.

10.4 Adaptive Enhancement Verification (preserve-originality mode)

The adaptive measurement was exercised on clean, blurred, noisy and veiled copies of the same sample photos, and the resulting pixel-level differences are real measurements:

Input state	Measured deficiency / shade	Effect on output vs plain 2x upscale
Clean / sharp photo	deficiency 0.00 - 0.03, shade ~0	preserved byte-for-byte (mean pixel difference 0.0, 100% of "clean" rows identical)
Blurred (Gaussian, sigma 2.2)	deficiency 0.60 - 0.70	genuine restoration applied, blended with the upscale via alpha up to ~0.86
Noisy (Gaussian, sigma 7/255)	deficiency 0.19	denoising applied (mean pixel change up to ~4.7 on textured samples)
Washed / veiled photo	shade 0.50 - 1.00	tonal pass-band widened (e.g. dynamic range ~108 -> ~124 levels), veil removed
Clean photo with real deep shadows	shade ~0 (healthy range)	untouched — shadows never lifted, byte-for-byte identical
All channels, after restore	—	tone means re-aligned to the input within 0.5 / 255 (< 0.2%)

What this means: the app no longer "enhances" a clean photo into a different version of itself — it leaves clean originals untouched, removes blur/noise proportionally to how bad they are, removes washed-out shade only when a real veil is measured, never lifts real shadows, and preserves the original tones

throughout.

11. What the Project Includes

23. Complete application (main.py) with an 8-option menu.
24. Two CNN architectures with different complexity/quality trade-offs.
25. Randomized degradation pipeline for realistic low-quality inputs.
26. Sharp perceptual loss (L1 + SSIM + Laplacian) — the key to visibly enhanced output.
27. True tiled 2x super-resolution for arbitrarily sized photos.
28. Adaptive enhancement: clean inputs preserved, blur/noise removed in proportion to the measured deficiency, real shade removed without lifting shadows, tones always preserved (ENHANCE_ADAPTIVE).
29. Unsharp-mask sharpening with configurable, measurement-scaled strength.
30. Honest evaluation — PSNR, SSIM, bicubic baseline, JSON + charts.
31. Terminal + GUI interfaces (GUI with thread-safe background processing and a full-window original/enhanced layout).
32. 40 auto-generated synthetic sample images — works out of the box.
33. Pre-trained demo models (best_model_v1.pth, best_model_v2.pth).
34. Windows launchers (setup.bat, run.bat).
35. Complete documentation: README, viva Q&A, and this report.

12. Limitations

36. Trained on synthetic data by default — real-world photos have complex, non-uniform degradation; adding real images to data/train improves results substantially.
37. CPU inference is slower than GPU for very large images (tiling keeps it memory-safe, but a 4000x3000 photo can take a minute on CPU).
38. The adaptive deficiency score is estimated from global measurements — a photo with blur in one area and noise in another may be under- or over-restored locally. Per-region (patch-level) gating is future work.
39. No true GAN/perceptual hallucination of texture at the level of ESRGAN-class networks; this is a deliberate, lightweight trade-off.
40. Synthetic samples are low in number (40) — ideal for demos and learning, not production-grade generalization.

13. Future Advancements

Improvement	Description
GAN / adversarial training	Add a small PatchGAN discriminator so the generator hallucinates richer, sharper texture (ESRGAN-style) while keeping the architecture laptop-friendly.
4x & 8x super-resolution	Extend the PixelShuffle stages and tile pipeline for higher upscaling factors.
Real-world degradation modeling	Mix synthetic degradation with real paired data (Real-ESRGAN style) for robust on-photo performance.
Perceptual (VGG) loss	Replace/augment the Laplacian term with a VGG-feature loss for human-visible quality.
Region-aware adaptive gating	Apply the blur/noise deficiency estimate per patch/region so mixed-quality photos are restored locally instead of globally.
ONNX / Web export	Export the model to ONNX and serve it in a browser upload tool.
Video enhancement	Apply the tiled pipeline per-frame with temporal smoothing.
Larger training datasets	Integrate DIV2K / Flickr2K with an automatic downloader + license note.
Quantization & early-exit	Quantize weights (INT8) for faster CPU inference.

14. Conclusion

IMAGE ENHANCER demonstrates a complete, self-contained, laptop-friendly deep-learning pipeline for image restoration and 2x super-resolution. Unlike classical interpolation, the trained CNN both upscales and restores lost detail; the combination of an SSIM + high-frequency loss and a final sharpening pass makes the enhancement clearly visible. The adaptive stage ties the system together from the user's point of view: every photo is measured first, so clean originals are preserved exactly as they are, genuine blur/noise is removed in proportion to how bad it is, washed-out shade is removed only when a real veil is measured (never lifting real shadows), and the original tones are always kept. Both models are light enough to train and run on a normal CPU, the evaluation is honest (PSNR/SSIM vs bicubic, saved as JSON and charts), and the project is fully documented for learning and presentation purposes. With V2, the project already outperforms bicubic on both measures, and the design leaves a clear roadmap (GANs, 4x, real-world degradation, region-aware gating) for future work.

*End of Report — IMAGE ENHANCER: AI-Based Image Restoration and 2x
Super-Resolution Using CNN*